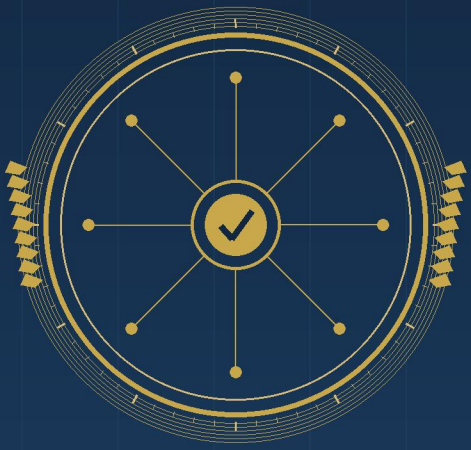

The Claude Certification Program

Practice Exams

Original Practice Questions & Full-Length Mock Exams
for the Claude Certified Architect — Foundations

برنامج شهادات Claude — نماذج الامتحانات

Dr. Fadhl Mohammed Alakwaa

د. فضل محمد الأكوع

English Edition

2026

The Claude Certification Program: Practice Exams

Original Practice Questions and Full-Length Mock Exams for the Claude Certified Architect — Foundations

برنامج شهادات Claude — نماذج الامتحانات

Dr. Fadhl Mohammed Alakwaa

د. فضل محمد الأكوع

First Edition — 2026

Fadhl Alakwaa Digital Library

This book was produced with the assistance of Claude Opus 4.8 (Anthropic).

أُنتج هذا الكتاب بمساعدة نموذج الذكاء الاصطناعي Claude Opus 4.8 من شركة Anthropic.

Fadhl Alakwaa Digital Library — First Edition 2026

Dedication

To every candidate who prepares not merely to pass, but to understand — who treats each practice question as a small laboratory for reasoning about agents, tools, and reliable systems. And to the learners of the Arab world building their path into this new profession one deliberate exercise at a time: may this book be a companion on the road to genuine mastery.

Important notice. The questions in this book are original practice items written to reflect the publicly documented structure, domains, and weightings of the Claude Certified Architect — Foundations examination. They are NOT actual examination questions, are not endorsed by or affiliated with Anthropic, and do not reproduce any confidential exam content. They are intended solely to help you rehearse the underlying concepts and the style of scenario-based reasoning the real exam rewards. Exam fees, question counts, domain weights, and passing thresholds cited here reflect the program as

documented in early 2026 and may change; always confirm current details on Anthropic's official certification pages before registering.

Contents

Introduction

A certification is only as valuable as the understanding it certifies, and understanding is built through practice. This book exists to give you that practice: a large bank of original, exam-style questions and a full-length mock examination designed around the published blueprint of the Claude Certified Architect — Foundations credential.

The companion volume in this series, "The Claude Certification Program: A Comprehensive Guide," lays out the map of the program — the four certifications, their prices and prerequisites, the domains and their weights, and the technical concepts each exam probes. This book assumes you have absorbed that map, or an equivalent understanding, and are now ready to test yourself against it. If a question here exposes a gap, treat that gap as a gift: it is far cheaper to discover it on these pages than in the proctored exam room.

The real Architect — Foundations exam presents sixty questions to be answered in one hundred and twenty minutes, scored on a scaled range from one hundred to one thousand, with seven hundred and twenty as the passing mark. Roughly a quarter of the weight falls on agentic architecture and orchestration, a fifth each on Claude Code workflows and on prompt engineering with structured output, and the remainder on tool design with the Model Context Protocol and on context

management and reliability. The questions are not trivia; they are scenarios that ask you to choose the design that a competent architect would choose, and to recognize the subtle traps that separate a working system from a fragile one.

This book mirrors that spirit. It is organized into five domain sets, each opening with a short reminder of what the domain tests and closing with fully explained answers. After the domain sets comes a complete mock exam of sixty questions, followed by its answer key and rationales, and finally a readiness self-assessment to help you decide whether you are prepared to register.

How to use this book. Work through one domain set at a time, under mild time pressure — aim for roughly two minutes per question, the pace the real exam demands. Resist the urge to peek at the answer before committing to a choice; the moment of commitment is where learning happens. When you read an explanation, do not stop at confirming your answer was correct. Read why each distractor is wrong, because the distractors are built from the plausible misconceptions that trip real candidates. Only when you can articulate why three options fail and one succeeds have you truly mastered a question.

A note on philosophy. Throughout these questions you will notice a recurring lesson: for anything that touches money, security, data

deletion, or safety, a competent architect enforces the constraint in code — through a hook, a gate, a validation layer, or a deterministic check — rather than merely asking the model to behave in a prompt. Prompts shape behavior; they do not guarantee it. This single principle, applied consistently, answers a surprising fraction of the hardest questions on the exam, and it is the difference between a demonstration and a production system. Keep it in mind as you work, and let the practice make it second nature.

May your preparation be deliberate and your understanding deep. And when you finally sit the exam, may it feel less like a test and more like a conversation you have already had many times before.

How to Use This Book

1. Set a timer. Give yourself two minutes per question; the real exam allows exactly that on average.
2. Commit before you check. Write down a letter for every question before reading any explanation.
3. Mine the distractors. For each wrong option, state in one sentence why a real candidate might pick it — and why it fails.
4. Track your domains. Note which of the five domains produces your errors; that is where to return.

5. Simulate once. Sit at least one full mock exam in a single uninterrupted session to build stamina.

PRACTICE SET

Domain 1 — Agentic Architecture & Orchestration

≈ 27% of the exam

> *This domain tests whether you can choose the right control structure for a task: when a fixed workflow beats an autonomous agent, how to compose orchestrator–worker and chaining patterns, how to bound agent loops, and — above all — when a constraint must be enforced in code rather than requested in a prompt.*

Q1. A team is building a system that classifies incoming support emails into one of five categories and routes each to a fixed downstream handler. The categories are stable and well defined. Which control structure is most appropriate?

A. A fully autonomous agent that decides its own steps on every email

B. A deterministic routing workflow that calls Claude once to classify, then dispatches by category

C. A multi-agent debate in which several agents argue about the category

D. An evaluator–optimizer loop that re-generates the category until confidence is maximal

Q2. In an orchestrator–worker pattern, a subagent is spawned to research a subtopic and return findings. The orchestrator later finds the subagent's answer references facts the orchestrator never sees. What is the most likely root cause?

A. The subagent hallucinated because subagents cannot use tools

B. The orchestrator and subagent share a context window, causing a collision

C. The subagent ran in an isolated context and its intermediate reasoning was not passed back explicitly

D. The model temperature was set too high for the orchestrator

Q3. A workflow must guarantee that no refund above $500 is ever issued without human approval. The refund action is a tool the model can call. What is the correct way to enforce this?

A. Add a strong instruction in the system prompt telling Claude never to refund more than $500 without approval

B. Provide few-shot examples of refusing large refunds

C. Gate the refund tool in code so that any amount over $500 is programmatically blocked and routed to human approval

D. Lower the temperature so the model behaves more predictably

Q4. Which situation most justifies an autonomous agent loop rather than a fixed prompt chain?

A. Summarizing a document into exactly three bullet points

B. Translating a paragraph from English to Arabic

C. Debugging a failing test suite where the number and order of steps cannot be known in advance

D. Extracting a date and an amount from an invoice

Q5. An agent loop occasionally runs indefinitely, calling tools in circles. Which safeguard most directly prevents runaway loops?

A. Increasing the model's maximum output tokens

B. A maximum-iteration cap plus a stop condition checked in the controlling code

C. Adding more detailed tool descriptions

D. Switching from named tool_choice to auto

Q6. A pipeline chains three prompts: extract → analyze → summarize. The analyze step frequently receives malformed input from the extract step. What is the best architectural fix?

A. Merge all three steps into one giant prompt

B. Add a validation gate between extract and analyze that checks the extracted structure before passing it on

C. Raise the temperature on the analyze step

D. Remove the extract step entirely

Q7. You need several independent subtasks (translate to three languages) done as fast as possible. Which orchestration pattern fits best?

A. Sequential prompt chaining, one language after another

B. Parallelization — issue the three independent calls concurrently and gather results

C. An evaluator–optimizer loop

D. A single autonomous agent that decides the order

Q8. In an evaluator–optimizer pattern, what is the role of the evaluator?

A. To generate the candidate output

B. To judge a candidate against explicit criteria and return feedback that drives the next revision

C. To route the request to the correct worker

D. To cache results for reuse

Q9. A guardrail must ensure an agent never accesses a production database in a customer-facing demo. The safest design is to:

A. Instruct the agent in its system prompt to avoid production

B. Only register non-production tools/credentials in the demo environment so production access is impossible

C. Ask the agent to confirm before each database call

D. Use a lower temperature during the demo

Q10. Which statement best distinguishes a 'workflow' from an 'agent' in Anthropic's framing?

A. Workflows use tools; agents never do

B. In a workflow the control flow is predefined by code; in an agent the model dynamically directs its own steps

C. Agents are always cheaper than workflows

D. Workflows require MCP; agents do not

Q11. An orchestrator delegates to five workers but the final synthesis is often incoherent. Which improvement targets the root cause most directly?

A. Give the orchestrator a structured synthesis step with an explicit schema for combining worker outputs

B. Increase each worker's max tokens

C. Convert the workers into a single monolithic prompt

D. Remove the orchestrator and let workers write to a shared file freely

Q12. For a task where a wrong action is catastrophic and irreversible (e.g., sending money), which combination is best?

A. High autonomy, no gates, strong prompt

B. Low autonomy with a code gate and a human-in-the-loop approval before the irreversible action

C. Maximum parallelization

D. An evaluator that grades the action after it executes

Q13. What is the primary reason to prefer smaller, composable sub-prompts over one enormous prompt in a complex pipeline?

A. Smaller prompts always produce longer outputs

B. Modularity makes each step testable, debuggable, and independently improvable

C. It is the only way to use tools

D. Large prompts are forbidden by the API

Q14. An agent must stop as soon as it has gathered enough evidence to answer. The cleanest way to express 'enough' is:

A. Hope the model stops on its own

B. Define an explicit, checkable stop condition in the loop (e.g., required fields all populated) and test it each iteration

C. Set temperature to zero

D. Add the word 'stop' to the system prompt

Q15. Which is a legitimate reason to introduce a routing step at the front of a system?

A. To send different request types to specialized prompts or models best suited to each

B. To increase token usage

C. Because routing is required by the Messages API

D. To disable tool use

Q16. A multi-agent system shares state by having every agent append to one growing context. As it scales, quality drops. The best remedy is:

A. Keep appending; larger context always helps

B. Give each agent a scoped context and pass only the specific data it needs, summarizing shared state deliberately

C. Remove all summaries

D. Raise temperature for every agent

Q17. Which design most reduces the blast radius when a single tool call fails inside an agent loop?

A. Let the exception crash the whole process

B. Return a structured error result to the model and let the loop decide whether to retry, choose another tool, or stop

C. Retry the same call forever

D. Ignore the failure silently and continue

Q18. You must add a hard spending cap across an entire agent session. Where does this cap belong?

A. In the system prompt as a polite request

B. In the orchestration code as a running token/cost accumulator that halts the session when the budget is hit

C. In the tool descriptions

D. In the few-shot examples

ANSWERS & EXPLANATIONS

Domain 1 — Agentic Architecture & Orchestration

Answer 1: B

When the set of outcomes is small, stable, and well defined, a single classification call feeding a deterministic dispatcher is the simplest thing that works — and simplicity is reliability. Autonomous agents, debates, and optimizer loops add latency, cost, and failure modes with no benefit here. The exam repeatedly rewards choosing the least-agentic structure that solves the problem; reserve autonomy for genuinely open-ended tasks.

Answer 2: C

Subagents run in their own isolated context. Only what the subagent explicitly returns crosses back to the orchestrator; its intermediate reasoning and any data it gathered but did not summarize are lost. This 'context isolation trap' is a favorite exam theme: if the orchestrator needs a fact, the subagent must place that fact in its returned output. Contexts are not shared (ruling out B), subagents can use tools (ruling out A), and temperature does not explain missing data (ruling out D).

Answer 3: C

Anything touching money, security, deletion, or safety must be enforced deterministically in code, not requested in a prompt. Prompts and examples shape behavior probabilistically but can be overridden by clever inputs or simply by the model erring. A code gate that inspects the amount before executing the refund is the only option that actually guarantees the rule. This 'enforce in code, don't ask in prompt' principle is the single most-tested idea in the exam.

Answer 4: C

Agent loops earn their complexity when the path to the goal is open-ended — the model must observe results, decide the next action, and iterate an unknown number of times, as in debugging. The other three tasks have a fixed, predictable shape and are better served by a single call or a short deterministic chain. Match the amount of autonomy to the amount of genuine uncertainty in the task.

Answer 5: B

Loop termination is a property of the controlling code, not of the prompt. A hard iteration cap combined with an explicit stop condition (goal met, or budget exhausted) guarantees the loop ends. Better descriptions may reduce confusion but cannot guarantee termination; token limits bound a single response, not the number of iterations; and tool_choice mode does not bound loop length.

Answer 6: B

Prompt chaining benefits from validation gates between links: a deterministic check that the output of one step meets the contract the next step expects, catching failures early and cheaply. Merging steps loses the modularity and debuggability that made chaining attractive; raising temperature worsens reliability; removing extract discards needed work. Gates between chain links are a core orchestration pattern.

Answer 7: B

When subtasks are independent and order does not matter, parallelization minimizes wall-clock latency by running them concurrently and aggregating. Sequential chaining needlessly serializes independent work; an evaluator–optimizer loop addresses quality refinement, not independent fan-out; and an autonomous agent adds decision overhead for a problem whose structure is already known.

Answer 8: B

The evaluator–optimizer loop pairs a generator that produces a candidate with an evaluator that scores it against criteria and returns actionable feedback; the generator then revises. It suits tasks with clear quality signals and room for iterative improvement, such as refining writing or code against a rubric. Generation, routing, and caching are other roles, not the evaluator's.

Answer 9: B

The strongest guardrail removes the capability entirely: if the demo environment simply lacks production credentials and tools, no prompt injection or model error can reach production. Prompt instructions and confirmation steps are advisory and bypassable. Controlling capability at the environment/tool-registration layer is deterministic and therefore the exam-preferred answer.

Answer 10: B

The defining difference is who owns the control flow. Workflows have paths that developers lay down in code — predictable and easy to test. Agents let the model decide the next step at runtime, trading predictability for flexibility. Both can use tools; neither is inherently cheaper; and MCP is orthogonal to the distinction.

Answer 11: A

Incoherent synthesis usually means the orchestrator lacks a disciplined way to merge heterogeneous worker outputs. Giving it an explicit schema and a dedicated synthesis step imposes structure on the combination. Bigger worker outputs do not help merging; collapsing to a monolith discards the pattern's benefits; and unstructured shared-file writes invite collisions and further incoherence.

Answer 12: B

Irreversible, high-stakes actions call for low autonomy, a deterministic gate, and human approval before execution. Grading after execution is too late for irreversible actions; parallelization is irrelevant to the risk; and high autonomy with only a prompt provides no real guarantee. Enforce the check before the point of no return.

Answer 13: B

Decomposition buys modularity: each step has a clear contract, can be tested in isolation, and can be improved without destabilizing the rest. This mirrors good software design. The other options are false — large prompts are allowed, tools work at any size, and output length is not determined by prompt size.

Answer 14: B

A loop should terminate on an explicit, code-checked condition — for instance, all required output fields are present and validated. Relying on the model to self-terminate is unreliable; temperature affects sampling, not stopping; and a keyword in the prompt is not a guarantee. Termination logic belongs in the controlling code.

Answer 15: A

Routing classifies an incoming request and dispatches it to the handler best suited for it — a simple model for easy cases, a stronger one or a specialized prompt for hard cases. This improves both quality and cost. The other options are false or irrelevant; routing is a design choice, not an API requirement, and it does not aim to inflate tokens or disable tools.

Answer 16: B

Unbounded shared context dilutes relevance and invites distraction and cost blow-up. Scoping each agent's context and passing only what it needs — with deliberate summarization of shared state — keeps each agent focused. More raw context is not automatically better; removing summaries and raising temperature both worsen coherence.

Answer 17: B

Robust agents treat tool failures as data: the tool returns a structured error the model can read and reason about, then the loop can retry with backoff, switch strategies, or halt gracefully. Crashing removes all recovery; infinite retries can loop forever; silent failure corrupts downstream reasoning. Structured, observable errors are the reliable middle path.

Answer 18: B

A budget cap is a deterministic constraint and must live in code: an accumulator that tracks spend across calls and stops the session at the threshold. Prompts, descriptions, and examples cannot guarantee a numeric limit is respected. This is the same 'enforce critical constraints in code' principle applied to cost.

PRACTICE SET

Domain 2 — Claude Code Configuration & Workflows

≈ 20% of the exam

> *This domain tests fluency with Claude Code as an engineering tool: how project and personal configuration differ and combine, how hooks and permissions enforce behavior deterministically, how MCP servers plug in, and how to run Claude Code non-interactively in automation and CI.*

Q1. A repository needs coding conventions that apply to everyone who works in it, checked into version control. Where should they live?

A. In each developer's personal user-level configuration

B. In a project-level CLAUDE.md committed to the repository

C. In a comment at the top of one source file

D. Only in the system prompt at runtime

Q2. You want personal shortcuts and preferences that follow you across every project on your machine but are never shared with the team. These belong in:

A. The project CLAUDE.md

B. Your user-level (personal) configuration

C. A .env file committed to the repo

D. The README

Q3. A team wants to guarantee that a linter runs and must pass before Claude Code is allowed to commit. The correct mechanism is:

A. A sentence in CLAUDE.md asking Claude to remember to lint

B. A hook that runs the linter at the appropriate lifecycle point and blocks the action on failure

C. A few-shot example of linting

D. Setting a low temperature

Q4. When both a project CLAUDE.md and a personal configuration define an instruction that conflicts, what is the safest mental model for a candidate?

A. Personal settings are ignored entirely inside a project

B. The layers combine, with more specific/local scope generally taking precedence; know the hierarchy rather than assuming

C. Project settings are always ignored

D. Whichever file is larger wins

Q5. You need Claude Code to run inside a CI pipeline with no interactive terminal, producing output another script can consume. You should:

A. Run it only in interactive mode and screen-scrape the terminal

B. Use its headless/non-interactive mode designed for scripting and automation

C. Ask it in the prompt to pretend there is no user

D. Disable all tools

Q6. A security policy forbids Claude Code from ever running a destructive shell command such as a recursive delete. The most reliable control is:

A. A note in CLAUDE.md

B. A permission/allow-list configuration (and/or a hook) that denies the dangerous command class

C. Trusting the model's judgment

D. Lowering temperature

Q7. What is the main purpose of adding an MCP server to a Claude Code setup?

A. To lower the model temperature

B. To give Claude Code standardized access to external tools, data, or systems through a common protocol

C. To replace the CLAUDE.md file

D. To disable hooks

Q8. A custom repeatable operation (e.g., 'open a PR following our template') should be exposed to the team as:

A. A tribal-knowledge Slack message

B. A reusable custom slash command / command file checked into the project

C. A one-off prompt each person retypes

D. A comment in unrelated code

Q9. Which best describes a good use of a subagent inside Claude Code?

A. Isolating a well-scoped subtask (e.g., focused code review) so it works in its own context and returns a concise result

B. Sharing the exact same context as the main agent to avoid passing data

C. Guaranteeing lower cost regardless of task

D. Bypassing all permissions

Q10. You want Claude Code to always load key architectural facts about a large codebase at the start of every session in that repo. The idiomatic place is:

A. Paste them manually every time

B. The project CLAUDE.md memory file

C. A binary file the model cannot read

D. The commit message of the latest change

Q11. A hook is configured to run tests after edits. Tests now fail intermittently and block all work. The best response is:

A. Delete the concept of hooks entirely from the product

B. Fix or scope the hook (e.g., target relevant tests, handle flakiness) so it enforces the intended gate without false blocks

C. Ignore all test failures forever

D. Move the tests into CLAUDE.md as prose

Q12. Which statement about Claude Code configuration precedence is the safest to rely on?

A. There is only ever one configuration file

B. Configuration is layered (enterprise/project/user and command-line), and understanding the order of precedence matters

C. Command-line flags are always ignored

D. Personal config overrides everything including explicit runtime flags

Q13. For a workflow that must behave identically for every teammate and in CI, you should prefer:

A. Personal, machine-specific settings

B. Project-committed configuration (CLAUDE.md, commands, hooks) so behavior is versioned and shared

C. Verbal instructions in standup

D. Undocumented shell aliases on one laptop

Q14. What is the best reason to keep secrets (API keys) out of CLAUDE.md?

A. CLAUDE.md is committed and shared, so secrets there would leak to everyone with repo access

B. CLAUDE.md cannot contain text

C. Secrets make the model slower

D. There is no reason; secrets belong there

Q15. You want Claude Code to run a specific formatting command every time it saves a file, deterministically. Use:

A. A hope-based prompt reminder

B. A hook bound to the relevant lifecycle event that runs the formatter

C. A random slash command

D. A larger context window

ANSWERS & EXPLANATIONS

Domain 2 — Claude Code Configuration & Workflows

Answer 1: B

Project-level memory (a CLAUDE.md committed to the repo) is shared by everyone who works in that repository and travels with the code. Personal/user-level config is private to one developer and does not propagate to teammates. A file-top comment is not read as configuration, and runtime-only instructions are not persistent or shared.

Answer 2: B

User-level configuration is scoped to you and applies across all your projects without being committed to any team repo. The project CLAUDE.md is shared; committing personal preferences there imposes them on everyone. A committed .env leaks to the team, and the README is documentation, not configuration.

Answer 3: B

Hooks are deterministic lifecycle triggers: they run your command at defined points and can block the operation if it fails. That makes them the right tool to enforce 'lint must pass before commit.' A reminder in CLAUDE.md is advisory and can be missed; examples and temperature do not enforce a gate. This is the Claude Code expression of 'enforce in code, don't ask in prompt.'

Answer 4: B

Configuration is layered, and the precedence follows a defined hierarchy in which more specific scopes generally override broader ones. The exam expects you to understand that the layers merge and to reason about precedence rather than assume one side is simply discarded. File size is never the deciding factor.

Answer 5: B

Claude Code offers a headless (non-interactive) mode intended precisely for automation and CI, where it runs a task and emits output that downstream scripts can parse. Interactive mode expects a human; screen-scraping is brittle; prompt pretense does not change the runtime mode; and disabling tools would cripple the task rather than enable automation.

Answer 6: B

Claude Code's permission system lets you constrain which commands and tools are allowed; combined with hooks, it can deny a dangerous command class outright. That is a deterministic control. A CLAUDE.md note and 'trust the model' are advisory; temperature is unrelated. Capability restriction beats behavioral request for anything destructive.

Answer 7: B

The Model Context Protocol standardizes how an assistant connects to external tools and data sources; adding an MCP server to Claude Code extends its reach to those systems through a common interface. It is unrelated to temperature, does not replace project memory, and does not disable hooks.

Answer 8: B

Custom commands (checked into the project) capture a repeatable operation once and make it reusable and consistent for the whole team. Slack messages and retyped prompts are error-prone and unshared; a stray comment is undiscoverable. Encoding workflows as commands is the maintainable pattern.

Answer 9: A

Subagents shine when a subtask is well scoped and benefits from its own isolated context, returning a focused result — code review is a classic example. Their context is isolated, not shared, so relevant data must be passed or returned explicitly. They do not inherently reduce cost for every task and do not bypass permissions.

Answer 10: B

The project memory file (CLAUDE.md) is loaded as context for work in that repository, making it the right home for durable architectural facts the assistant should always know. Manual pasting is tedious and inconsistent; a binary file is unreadable as context; and commit messages are not loaded as standing project memory.

Answer 11: B

The hook is doing its job — enforcing a gate — but it is mis-scoped or catching flaky tests. The fix is to refine the hook: narrow it to relevant tests and handle flakiness, preserving the guarantee while removing false blocks. Abandoning hooks or ignoring failures discards the safety; prose in CLAUDE.md cannot run tests.

Answer 12: B

Claude Code reads configuration from multiple layers and resolves them by a defined precedence; a competent user reasons about that order rather than assuming a single file or a fixed winner. Runtime flags are not universally ignored, and personal config does not trump explicit higher-precedence signals.

Answer 13: B

Reproducible, shared behavior comes from configuration committed to the project and versioned alongside the code — memory, commands, and hooks. Personal settings, verbal instructions, and per-laptop aliases do not propagate and cannot be relied on in CI.

Answer 14: A

Project memory is checked into the repository and shared with everyone who can read it, so placing secrets there exposes them broadly and permanently in history. Secrets belong in a secure secret store or environment configuration, never in shared, versioned memory.

Answer 15: B

Hooks bind commands to lifecycle events and run them deterministically — exactly what 'format on every save' requires. Prompt reminders are advisory and skippable; an unrelated slash command does not trigger automatically; context size is irrelevant to running a formatter.

PRACTICE SET

Domain 3 — Prompt Engineering & Structured Output

≈ 20% of the exam

> *This domain tests reliable prompting technique — few-shot examples, XML structuring, chain-of-thought, prefilling, system prompts — and, crucially, how to obtain machine-readable structured output you can trust, most robustly by driving structure through tool use.*

Q1. You need Claude to return data your program can parse with certainty into fixed fields. The most robust approach is:

A. Ask for JSON in prose and hope the format holds

B. Define a tool whose input schema is the desired structure and let the model populate it

C. Request the answer 'as neatly as possible'

D. Increase temperature to encourage creativity

Q2. Few-shot examples are most valuable when:

A. The task format or style is easier to show than to describe

B. You want to increase randomness

C. You need to hide the task from the model

D. The task is a single arithmetic operation

Q3. Why wrap distinct parts of a prompt (instructions, context, examples) in XML-style tags?

A. It is required by the API

B. Clear delimiters reduce ambiguity and help the model attend to the right section

C. It lowers cost automatically

D. It disables tool use

Q4. You want the model to reason step by step before answering a hard problem, but you need only the final answer in the output. A clean technique is:

A. Forbid all reasoning

B. Let it think in a designated section (e.g., a <thinking> block) and place the final answer in a separate, clearly delimited section

C. Raise temperature to force reasoning

D. Ask it to answer instantly with no thought

Q5. Prefilling the start of the assistant's response is useful primarily to:

A. Increase the temperature

B. Steer format or skip preamble by beginning the reply with a chosen token (e.g., '{' to push straight into JSON)

C. Delete the system prompt

D. Disable stop sequences

Q6. The best role for the system prompt is to:

A. Hold the user's specific question

B. Establish durable role, rules, tone, and constraints that persist across the conversation

C. Store the final answer

D. Contain random noise to improve diversity

Q7. A JSON-producing prompt occasionally emits a friendly sentence before the JSON, breaking your parser. The most reliable fixes include:

A. Only asking more politely

B. Using tool-based structured output, and/or prefilling '{' and instructing that the response contain JSON only

C. Raising temperature

D. Removing the schema

Q8. When should you lower temperature toward zero?

A. When you want deterministic, consistent outputs such as classification or extraction

B. When you want maximum creative variety

C. Only for translation

D. Temperature has no effect on determinism

Q9. You must extract five specific fields from messy documents with high reliability. Best design:

A. Freeform prose request at high temperature

B. A tool/schema with the five typed fields, low temperature, and clear instructions on handling missing values

C. Ask for a poem summarizing the document

D. Rely on the model to guess the fields it thinks matter

Q10. What is a correct use of stop sequences?

A. To make the model more creative

B. To halt generation when a specified delimiter appears, bounding the output cleanly

C. To increase the context window

D. To enforce a spending cap

Q11. Chain-of-thought reasoning tends to help most on:

A. Simple lookups and single-step facts

B. Multi-step problems (math, logic, complex analysis) where intermediate steps reduce errors

C. Tasks where any reasoning is harmful

D. Lowering latency

Q12. You provide few-shot examples but the model over-imitates a superficial quirk of them (e.g., always answering 'Yes'). The best fix is:

A. Add more examples with that same quirk

B. Diversify and balance the examples so they represent the true distribution and edge cases

C. Remove all instructions

D. Raise temperature to 1.5

Q13. To make a long, information-dense prompt more reliable, a strong practice is:

A. Place the key question and instructions clearly (often near the end) and structure context with tags

B. Bury the question randomly in the middle of unlabeled text

C. Remove all structure

D. Use only emojis as delimiters

Q14. Which is the strongest guarantee that output conforms to a required structure?

A. A polite request for that structure

B. Tool-based structured output whose schema encodes the structure, validated by your code on receipt

C. A high temperature

D. A longer system prompt with no schema

Q15. You want consistent tone across a support bot's replies. The best place to specify tone is:

A. In each user's message

B. In the system prompt, as a persistent instruction

C. In a stop sequence

D. In the temperature value

ANSWERS & EXPLANATIONS

Domain 3 — Prompt Engineering & Structured Output

Answer 1: B

Driving structured output through a tool's input schema is the most reliable way to get well-formed, typed data: the schema constrains the shape, and your code receives structured input rather than free text to parse. Asking for JSON in prose works often but not always; vague requests and higher temperature reduce reliability. When structure must be guaranteed, prefer tool-based structured output.

Answer 2: A

Few-shot prompting shines when demonstrating the desired input→output mapping communicates format, tone, or edge-case handling more clearly than an abstract description. It reduces ambiguity by example. It is not about randomness or concealment, and trivial single operations rarely need examples.

Answer 3: B

Tagged sections give the model unambiguous structure — 'this is context, this is the question, these are examples' — improving reliability, especially in long prompts. Tags are a convention, not an API requirement; they neither reduce cost by themselves nor affect tool use.

Answer 4: B

Encouraging explicit reasoning improves accuracy on hard tasks; directing that reasoning into a designated block and the final answer into a separate delimited section lets you extract just the answer while still gaining the benefit of the reasoning. Forbidding reasoning or demanding instant answers sacrifices accuracy; temperature does not create structured reasoning.

Answer 5: B

Prefilling seeds the assistant turn with initial text, nudging the model to continue in a chosen format — starting with '{' to encourage immediate JSON, for example, or skipping conversational preamble. It does not change temperature, remove the system prompt, or affect stop sequences.

Answer 6: B

The system prompt sets the model's standing role, guardrails, tone, and persistent constraints — the frame within which each user turn is interpreted. The specific question belongs in the user message; the answer is generated, not stored in the system prompt; and noise degrades quality.

Answer 7: B

To eliminate stray preamble, either drive the output through a tool schema (structured by construction) or prefill the opening brace and instruct 'JSON only.' Politeness does not constrain format; higher temperature increases variability; removing the schema removes the very structure you need.

Answer 8: A

Low temperature yields more deterministic, repeatable outputs, which is what classification, extraction, and format-critical tasks want. Higher temperature suits creative variety. Temperature does affect determinism, and its use is not limited to translation.

Answer 9: B

Reliable field extraction combines a typed schema (naming exactly the five fields), low temperature for consistency, and explicit rules for missing data. Freeform high-temperature output is inconsistent; a poem is the wrong format; and letting the model choose fields abandons the contract your system depends on.

Answer 10: B

Stop sequences end generation as soon as a specified string appears, which is handy for bounding output at a known delimiter. They do not affect creativity, context size, or spending. (Spending caps live in your orchestration code.)

Answer 11: B

Explicit intermediate reasoning pays off on multi-step problems, where working through steps reduces mistakes. For trivial lookups it adds latency with little benefit, and it does not reduce latency. Reasoning is generally helpful, not harmful, on hard tasks.

Answer 12: B

Models learn patterns from examples, including unintended ones. Balancing and diversifying the examples so they reflect the real distribution and include edge cases corrects the skew. Reinforcing the quirk worsens it; removing instructions loses guidance; extreme temperature destabilizes output.

Answer 13: A

Long prompts benefit from clear structure — tagged context and a clearly placed instruction/question the model can latch onto. Burying the question in unlabeled text invites the model to lose track of it. Structure aids reliability; emojis are not dependable delimiters.

Answer 14: B

Encoding the structure in a tool schema and validating the received input in code is the strongest guarantee: the model fills a defined shape and your code rejects anything malformed. Requests and long prompts help but do not guarantee; temperature is irrelevant to conformance.

Answer 15: B

Persistent behavioral qualities like tone belong in the system prompt, which frames every turn. Putting tone in each user message is repetitive and fragile; stop sequences and temperature do not encode tone.

PRACTICE SET

Domain 4 — Tool Design & MCP Integration

≈ 18% of the exam

> *This domain tests how to design tools the model can use well — precise descriptions and schemas, sensible tool_choice, structured error handling — and how the Model Context Protocol standardizes connections to external tools, data, and resources.*

Q1. The single most important factor in whether the model uses a tool correctly is:

A. The tool's internal implementation language

B. A clear, precise tool description and input schema that convey when and how to use it

C. The color of the log output

D. The temperature setting

Q2. You must force the model to call a specific tool on this turn. Set tool_choice to:

A. auto

B. the named tool (forcing that specific tool)

C. any

D. none of these can force a specific tool

Q3. tool_choice set to 'any' means:

A. The model may answer without any tool

B. The model must use one of the provided tools, but chooses which

C. The model must use every tool

D. Tools are disabled

Q4. A tool that fails should return, to the model:

A. Nothing; just crash

B. A structured error result describing what went wrong so the model can adapt

C. A misleading success message

D. An unrelated large blob of text

Q5. In the Model Context Protocol, the primary value proposition is:

A. A single vendor lock-in format

B. A standardized, reusable way to connect assistants to external tools, data, and resources

C. A replacement for prompts

D. A way to raise temperature

Q6. An MCP server exposes both 'resources' and 'tools.' The key distinction is that:

A. Resources are actions; tools are read-only data

B. Tools are callable actions/functions; resources are data/content the client can read

C. They are identical

D. Resources can never be used by a model

Q7. A tool description says only 'gets stuff.' The model misuses it constantly. The fix is to:

A. Lower temperature

B. Rewrite the description to state precisely what it does, when to use it, and what parameters mean

C. Add more tools with vague names

D. Force tool_choice to none

Q8. For a tool that will be retried automatically on failure, a valuable design property is:

A. Non-determinism

B. Idempotency, so repeated calls with the same input do not cause duplicate side effects

C. Silent partial writes

D. Randomized outputs

Q9. You provide several tools; the model keeps answering from memory instead of calling any tool when it should. A reasonable intervention is:

A. Set tool_choice to 'any' (or the specific tool) for turns where a tool is required, and sharpen descriptions

B. Delete all tools

C. Raise max tokens

D. Add a stop sequence

Q10. When the model returns a tool_use request, the correct next step in the loop is to:

A. Ignore it and ask another question

B. Execute the tool, then return a tool_result with the output so the model can continue

C. End the conversation

D. Lower the temperature

Q11. Designing tool input schemas, you should:

A. Use vague, untyped free-text parameters everywhere

B. Specify typed parameters with clear names and descriptions, marking which are required

C. Avoid descriptions to save tokens

D. Randomize parameter order each call

Q12. Multiple independent tool calls are needed and the API supports issuing them together. Doing so primarily improves:

A. Determinism of the model

B. Latency/efficiency by handling independent calls in parallel rather than one round-trip each

C. The passing score

D. The temperature

Q13. A safe default when unsure whether the model should use tools on a given turn is:

A. tool_choice = auto, letting the model decide based on the request and tool descriptions

B. Always force every tool

C. Disable tools permanently

D. Set temperature to 2

Q14. An MCP integration returns errors as plain crashes that terminate the session. Improve reliability by:

A. Returning structured error envelopes the model can read and handle gracefully

B. Hiding the errors

C. Retrying infinitely with no cap

D. Increasing output tokens

Q15. You expose a 'delete_record' tool. To prevent accidental mass deletion, the best safeguard is:

A. A prompt asking the model to be careful

B. Code-side validation/limits (e.g., require an id, cap batch size, require confirmation for bulk deletes) enforced before execution

C. A friendlier tool name

D. Higher temperature

ANSWERS & EXPLANATIONS

Domain 4 — Tool Design & MCP Integration

Answer 1: B

The model decides whether and how to call a tool based on its description and schema. Precise, unambiguous descriptions — what the tool does, when to use it, what each parameter means — are the dominant factor in correct tool use. Implementation language, logging, and temperature are secondary or irrelevant to selection.

Answer 2: B

Naming a tool in tool_choice forces the model to call exactly that tool. 'auto' lets the model decide whether to use any tool; 'any' forces it to use some tool but not which one. So to compel one specific tool, specify it by name.

Answer 3: B

'any' requires the model to call some tool while leaving the selection to the model. It does not permit a tool-free answer ('auto' allows that), does not require calling all tools, and does not disable tools.

Answer 4: B

Returning a structured, informative error lets the model reason about the failure and choose to retry, switch tools, or ask for help. Crashing removes recovery, faking success corrupts the reasoning, and irrelevant output wastes context and confuses the model. Design tools to fail informatively.

Answer 5: B

MCP provides a common protocol so that tools, data sources, and resources can be exposed once and consumed by any MCP-aware client — reducing bespoke integrations. It is an open standard rather than lock-in, does not replace prompting, and has nothing to do with temperature.

Answer 6: B

In MCP, tools are invocable functions that do something, while resources are addressable data or content the client can read into context. They are distinct concepts, not identical, and resources are meant to be consumed. (The exact wiring is implementation-specific, but the action-vs-data distinction is the point.)

Answer 7: B

Vague descriptions cause misuse. The remedy is a precise description covering purpose, appropriate use, and parameter semantics. Temperature will not fix ambiguity; more vague tools worsen confusion; disabling tools abandons the capability.

Answer 8: B

If a tool may be retried, idempotency ensures that calling it again with the same input is safe and does not, say, charge a card twice. Non-determinism, silent partial writes, and randomized outputs all make retries dangerous. Design retryable tools to be idempotent.

Answer 9: A

If the model should use a tool but does not, you can force tool use with 'any' (or name the required tool) and improve the descriptions so the model recognizes when tools apply. Deleting tools removes the capability; token limits and stop sequences do not affect tool selection.

Answer 10: B

The tool-use loop requires you to run the requested tool and feed the outcome back as a tool_result, letting the model incorporate it and proceed. Ignoring or ending breaks the loop; temperature is irrelevant to this mechanic.

Answer 11: B

Good schemas use clear, typed, well-described parameters and mark required fields, guiding the model to supply correct inputs. Vague untyped parameters and missing descriptions invite errors; parameter order should be stable and meaningful.

Answer 12: B

Requesting independent tool calls in parallel reduces the number of sequential round-trips and improves latency and efficiency. It does not change the model's determinism, your exam score, or temperature.

Answer 13: A

'auto' is the balanced default: the model uses a tool when the request and descriptions warrant it, and answers directly otherwise. Forcing all tools or disabling them removes that judgment; extreme temperature is unrelated.

Answer 14: A

Structured error envelopes turn failures into information the model can act on — retry, fall back, or report — instead of hard crashes. Hiding errors corrupts reasoning, uncapped retries can loop forever, and token limits are irrelevant to error handling.

Answer 15: B

Destructive tools need deterministic safeguards in code: require specific identifiers, cap batch sizes, and demand confirmation for bulk operations — all checked before anything is deleted. Prompts, names, and temperature cannot guarantee safety for irreversible actions.

PRACTICE SET

Domain 5 — Context Management & Reliability

≈ 15% of the exam

> *This domain tests how to keep systems accurate and affordable at scale: budgeting the context window, using prompt caching and retrieval, summarizing long histories, reading stop_reason, retrying wisely, and choosing batch processing when latency can be traded for cost.*

Q1. A conversation grows until it approaches the context-window limit and quality degrades. A sound strategy is:

A. Ignore it; models handle infinite context

B. Summarize or compact older turns and retain only what is relevant, keeping the window focused

C. Raise temperature

D. Delete the system prompt

Q2. You repeatedly send the same large, unchanging preamble (e.g., a long policy document) across many requests. To cut cost and latency, use:

A. Prompt caching of the stable prefix so it need not be reprocessed each time

B. A higher temperature

C. A different font

D. More tools

Q3. A job processes millions of documents overnight where immediate responses are unnecessary. The cost-efficient choice is:

A. Real-time single requests at peak priority

B. The Batch API, trading turnaround (up to ~24h) for a significant per-token discount

C. Increasing temperature

D. Sending everything in one giant prompt

Q4. Your code must detect when the model stopped because it hit the output-token limit versus finishing naturally. You inspect:

A. The temperature

B. The stop_reason field of the response

C. The system prompt

D. The tool descriptions

Q5. To keep answers grounded in a large knowledge base without exceeding the context window, use:

A. Paste the entire knowledge base every time

B. Retrieval — fetch only the relevant passages for each query and include those

C. Raise temperature

D. Remove all context

Q6. A transient network error causes an API call to fail. The reliable pattern is:

A. Crash the whole pipeline

B. Retry with exponential backoff up to a sensible cap, then fail gracefully

C. Retry instantly forever

D. Ignore and pretend it succeeded

Q7. When a response's stop_reason indicates the output was truncated at max tokens, a good handling is:

A. Assume the answer is complete

B. Continue generation (e.g., request the remainder) or raise the token budget, then stitch the result

C. Lower temperature and resend the same request unchanged

D. Delete the conversation

Q8. Which practice most improves cost-efficiency for a chatbot with very long sessions?

A. Never summarizing and always sending full history

B. Periodically compacting history into a running summary and dropping stale detail

C. Maximizing temperature

D. Adding unused tools

Q9. You need reproducible outputs for a regression test suite. Alongside fixed inputs you should:

A. Use a high temperature

B. Use a low/zero temperature to reduce sampling variability

C. Randomize the system prompt

D. Disable stop_reason

Q10. A reliability review finds the system silently drops tool errors. The most important change is:

A. Surface errors as structured, observable results and handle them explicitly in the loop

B. Increase the context window

C. Add more few-shot examples

D. Raise temperature

Q11. Which is the best reason to prefer retrieval over dumping a whole corpus into context?

A. It always increases temperature

B. It keeps the prompt focused and within limits while improving grounding and cost

C. It disables tools

D. It removes the need for any prompt

Q12. You must guarantee a per-request output never exceeds a length your UI can render. You should:

A. Only ask nicely in the prompt

B. Set an appropriate max_tokens and handle truncation via stop_reason, optionally instructing brevity

C. Raise temperature

D. Remove the system prompt

Q13. For a workload mixing urgent user chats and a huge nightly analytics run, a sensible split is:

A. Run everything in real time

B. Serve chats in real time and send the analytics run through the Batch API for the discount

C. Batch the urgent chats

D. Use one giant prompt for both

Q14. The cleanest signal that the model wants to call a tool (so your loop should execute one) is:

A. A drop in temperature

B. A stop_reason indicating a tool use, with the corresponding tool_use block in the response

C. An empty system prompt

D. A longer context window

Q15. To keep a long-running agent both accurate and affordable, the best combined practice is:

A. Never summarize, never cache, never retrieve

B. Cache stable prefixes, retrieve only relevant context, compact history, and batch latency-tolerant work

C. Maximize temperature and tokens on every call

D. Send the full transcript at maximum length each turn

ANSWERS & EXPLANATIONS

Domain 5 — Context Management & Reliability

Answer 1: B

Context windows are finite, and stuffing them dilutes relevance. Summarizing or compacting older history while keeping salient facts maintains focus and quality. Models do not have infinite context; temperature and removing the system prompt do not solve the size problem.

Answer 2: A

Prompt caching lets a large, stable prefix be reused across requests without reprocessing it every time, reducing cost and latency for repeated context. Temperature, fonts, and tool count do not address repeated-context cost.

Answer 3: B

Batch processing is designed for large, latency-tolerant workloads: you accept a longer turnaround (commonly up to about a day) in exchange for a substantial discount. Real-time requests cost more for no benefit here; temperature is irrelevant; a single giant prompt risks context limits and loses parallelism.

Answer 4: B

The stop_reason indicates why generation ended — natural end, a stop sequence, a tool-use request, or hitting max tokens — so your code can react (e.g., continue a truncated response). Temperature, system prompt, and tool descriptions do not convey stop cause.

Answer 5: B

Retrieval brings in just the passages relevant to the current query, keeping the window focused and grounded while staying within limits. Pasting everything wastes context and money and may exceed limits; removing context ungrounds the answer; temperature is irrelevant.

Answer 6: B

Transient failures call for bounded retries with exponential backoff, then a graceful fallback if they persist. Crashing is brittle, instant infinite retries can hammer the service and loop forever, and pretending success corrupts results.

Answer 7: B

Truncation means the model had more to say; you should continue the generation or increase the token budget and reassemble the full output. Assuming completeness loses content; blindly resending unchanged repeats the truncation; deleting the conversation discards work.

Answer 8: B

Long sessions grow expensive if you resend everything. Periodic compaction into a running summary keeps essential state while trimming token count and preserving quality. Sending full history is costly; temperature and unused tools do not help cost.

Answer 9: B

Low or zero temperature minimizes sampling variability, giving more consistent outputs suitable for regression testing. High temperature and randomized prompts increase variance; stop_reason is a read-only signal, not a knob for reproducibility.

Answer 10: A

Silent error-dropping is a reliability hazard. Making errors structured and observable, and handling them explicitly (retry, fallback, alert), is the key fix. Window size, examples, and temperature do not address swallowed errors.

Answer 11: B

Retrieval supplies only the relevant slice, which keeps the prompt focused, respects context limits, and reduces cost while improving grounding. It does not affect temperature, disable tools, or eliminate the prompt.

Answer 12: B

A hard output bound is enforced by max_tokens, with stop_reason letting you detect and handle truncation; prompt instructions to be brief help but are not a hard cap. Temperature and removing the system prompt do not bound length.

Answer 13: B

Match the processing mode to the latency need: interactive chats require real-time responses, while the latency-tolerant nightly analytics is ideal for the discounted Batch API. Batching urgent chats would harm user experience; a single giant prompt is neither.

Answer 14: B

When the model requests a tool, the response carries a tool_use block and stop_reason reflects tool use; your loop reads that and executes the tool. Temperature, system-prompt state, and window size are not the signal.

Answer 15: B

Accuracy-plus-affordability at scale comes from combining the levers: cache stable prefixes, retrieve only what is relevant, compact history, and route latency-tolerant work to batch. The other options inflate cost or degrade quality.

Mock Exam A

Full-Length — 60 Questions

Mock Exam A

This is a full-length mock examination of sixty questions, mirroring the count and the two-hour window of the real Architect — Foundations exam. Sit it in one uninterrupted session. Mark your answers first; the answer key with rationales follows the questions. Aim to finish within one hundred and twenty minutes, and afterward compute your score as the percentage correct, noting which domains produced your misses.

1. A startup wants Claude to file expense reimbursements automatically. Reimbursements over $1,000 must never go out without a manager's approval. What guarantees the rule?

A. A firm instruction in the system prompt

B. A code gate that blocks amounts over $1,000 and routes them to approval

C. Few-shot examples of asking for approval

D. A low temperature setting

2. A task classifies tickets into three fixed queues. Which is the simplest sufficient design?

A. An autonomous agent with open-ended tools

B. One classification call feeding a deterministic dispatcher

C. A five-agent debate

D. An evaluator–optimizer loop

3. An orchestrator spawns a research subagent but never sees a fact the subagent used. Why?

A. Subagents cannot use tools

B. The subagent's isolated context wasn't passed back explicitly

C. Temperature too high

D. The API dropped the message

4. You need three independent summaries generated as fast as possible. Best pattern?

A. Sequential chaining

B. Parallelization of the independent calls

C. Evaluator–optimizer

D. A single agent choosing order

5. A repo needs conventions everyone shares and versions. Where do they go?

A. Personal user config

B. A project CLAUDE.md committed to the repo

C. A code comment

D. Runtime-only note

6. You must force the model to use one particular tool this turn. Set tool_choice to:

A. auto

B. the named tool

C. any

D. none

7. A tool call fails mid-loop. Best behavior?

A. Crash the process

B. Return a structured error the model can act on

C. Retry forever

D. Continue silently

8. A conversation nears the context limit and degrades. Do what?

A. Nothing; context is infinite

B. Summarize/compact older turns, keep relevant state

C. Raise temperature

D. Drop the system prompt

9. Millions of documents must be processed by morning; latency doesn't matter. Use:

A. Real-time requests

B. The Batch API for the discount

C. Higher temperature

D. One giant prompt

10. You need parseable, typed fields from messy text with high reliability. Best approach?

A. Ask for JSON in prose

B. Drive output through a tool schema and validate in code

C. Ask 'as neat as possible'

D. Raise temperature

11. A linter must pass before Claude Code commits. Enforce with:

A. A CLAUDE.md reminder

B. A hook that runs the linter and blocks on failure

C. A few-shot example

D. Low temperature

12. An agent loop sometimes never ends. The fix that guarantees termination?

A. Bigger max tokens

B. A max-iteration cap plus an explicit stop condition in code

C. Better tool descriptions

D. tool_choice = auto

13. You send the same long policy prefix on every call. Cut cost with:

A. Prompt caching of the stable prefix

B. Higher temperature

C. More tools

D. A new font

14. Your code must know the model stopped at the token limit. Inspect:

A. temperature

B. stop_reason

C. the system prompt

D. tool descriptions

15. A demo must be unable to touch production data. Strongest control?

A. A prompt telling the agent to avoid production

B. Register only non-production tools/credentials in the demo

C. Confirm before each call

D. Lower temperature

16. Few-shot examples help most when:

A. Format/style is easier to show than describe

B. You want randomness

C. You want to hide the task

D. The task is one addition

17. A destructive delete_record tool needs protection from mass deletion. Best safeguard?

A. A careful-sounding prompt

B. Code-side limits: require id, cap batch, confirm bulk — before execution

C. A friendlier name

D. Higher temperature

18. Personal shortcuts that follow you across all projects but never reach the team live in:

A. Project CLAUDE.md

B. User-level personal config

C. A committed .env

D. The README

19. A prompt chain's analyze step keeps getting malformed input. Best fix?

A. Merge all steps into one prompt

B. Add a validation gate between steps

C. Raise temperature

D. Delete the extract step

20. tool_choice = 'any' means:

A. May answer with no tool

B. Must use some provided tool, model picks which

C. Must use every tool

D. Tools disabled

21. A transient 503 error hits an API call. Reliable pattern?

A. Crash the pipeline

B. Exponential backoff retries up to a cap, then graceful failure

C. Instant infinite retries

D. Pretend success

22. You want step-by-step reasoning but only the final answer in output. Do what?

A. Forbid reasoning

B. Reason in a <thinking> block; put the answer in a separate delimited section

C. Raise temperature

D. Demand an instant answer

23. An MCP server is added to Claude Code mainly to:

A. Lower temperature

B. Give standardized access to external tools/data/resources

C. Replace CLAUDE.md

D. Disable hooks

24. A wrong action here is irreversible (wire transfer). Best combo?

A. High autonomy, prompt only

B. Low autonomy, code gate, human approval before execution

C. Max parallelization

D. Grade after execution

25. A JSON prompt sometimes adds a friendly preamble that breaks parsing. Robust fix?

A. Ask more politely

B. Tool-based structured output, or prefill '{' with 'JSON only'

C. Raise temperature

D. Remove the schema

26. An agent should stop once required fields are filled. Express 'enough' as:

A. Hope it stops

B. An explicit, code-checked stop condition each iteration

C. Temperature zero

D. The word 'stop' in the prompt

27. The dominant factor in correct tool use is:

A. Implementation language

B. A precise description and input schema

C. Log color

D. Temperature

28. You must bound output length for a UI. Enforce with:

A. A polite request only

B. max_tokens plus stop_reason handling

C. Higher temperature

D. Removing the system prompt

29. A tool may be auto-retried. Valuable design property?

A. Non-determinism

B. Idempotency

C. Silent partial writes

D. Randomized outputs

30. Route requests at the front of a system in order to:

A. Send each request type to the best-suited prompt/model

B. Increase tokens

C. Satisfy an API requirement

D. Disable tools

31. A CI pipeline needs Claude Code with no interactive terminal. Use:

A. Interactive mode + screen scraping

B. Headless/non-interactive mode

C. A prompt pretending no user exists

D. Disable all tools

32. Keep answers grounded in a huge KB without blowing the context window. Use:

A. Paste the whole KB each time

B. Retrieval of only relevant passages

C. Higher temperature

D. No context at all

33. Which distinguishes a workflow from an agent?

A. Workflows use tools; agents don't

B. Workflow control flow is coded; agents direct their own steps

C. Agents are always cheaper

D. Workflows require MCP

34. Secrets must not go in CLAUDE.md because:

A. It's committed and shared, leaking secrets

B. It can't hold text

C. Secrets slow the model

D. No reason

35. The model answers from memory when it should call a tool. Intervention?

A. Force tool use (any/named) and sharpen descriptions

B. Delete all tools

C. Raise max tokens

D. Add a stop sequence

36. Response was truncated at max tokens (per stop_reason). Handle by:

A. Assuming completeness

B. Continuing generation or raising the budget, then stitching

C. Resending unchanged at low temperature

D. Deleting the conversation

37. XML-style tags around prompt sections primarily:

A. Are required by the API

B. Reduce ambiguity and focus attention on the right section

C. Lower cost automatically

D. Disable tools

38. Independent tool calls issued together mainly improve:

A. Model determinism

B. Latency/efficiency via parallelism

C. The passing score

D. Temperature

39. A monolithic mega-prompt is hard to maintain. Prefer decomposition because:

A. Smaller prompts give longer output

B. Modularity makes steps testable and improvable

C. It's the only way to use tools

D. Big prompts are banned

40. Reproducible outputs for regression tests need:

A. High temperature

B. Low/zero temperature with fixed inputs

C. Randomized system prompt

D. Disabling stop_reason

41. An evaluator in an evaluator–optimizer loop:

A. Generates the candidate

B. Judges against criteria and returns feedback to drive revision

C. Routes to a worker

D. Caches results

42. A hook running the full test suite blocks work with flaky failures. Best response?

A. Remove hooks entirely

B. Scope/fix the hook (relevant tests, handle flakiness)

C. Ignore all failures

D. Move tests into CLAUDE.md prose

43. A support bot needs a consistent tone. Put it in:

A. Each user message

B. The system prompt

C. A stop sequence

D. The temperature

44. A subagent is a good fit for:

A. A well-scoped subtask in its own context returning a concise result

B. Sharing the main context to avoid passing data

C. Guaranteed lower cost always

D. Bypassing permissions

45. MCP resources vs tools:

A. Resources are actions; tools are data

B. Tools are callable actions; resources are readable data/content

C. They're identical

D. Resources can't be used

46. A runaway multi-agent system shares one ever-growing context and degrades. Fix?

A. Keep appending

B. Scope each agent's context; pass only needed data; summarize shared state

C. Remove all summaries

D. Raise every temperature

47. A spending cap across a session belongs:

A. In the system prompt as a request

B. In code as a cost accumulator that halts at the budget

C. In tool descriptions

D. In few-shot examples

48. The model returns a tool_use request. Next step?

A. Ignore and ask again

B. Execute the tool and return a tool_result to continue

C. End the conversation

D. Lower temperature

49. Config conflict between project and personal settings. Safe mental model?

A. Personal is always ignored in a project

B. Layers combine by a defined precedence; know the hierarchy

C. Project is always ignored

D. Larger file wins

50. Best reason to prefer retrieval over dumping a whole corpus?

A. It raises temperature

B. It keeps the prompt focused, within limits, grounded, and cheaper

C. It disables tools

D. It removes the prompt

51. Prefilling the assistant reply is used to:

A. Raise temperature

B. Steer format/skip preamble by seeding the first tokens

C. Delete the system prompt

D. Disable stop sequences

52. An MCP integration crashes the session on errors. Improve by:

A. Structured error envelopes the model can handle

B. Hiding errors

C. Uncapped infinite retries

D. More output tokens

53. Lower temperature toward zero when you want:

A. Deterministic outputs (classification/extraction)

B. Maximum creative variety

C. Only translation

D. No effect exists

54. A tool description reads 'gets stuff' and is misused. Fix?

A. Lower temperature

B. Rewrite it to state purpose, when to use, and parameter meanings

C. Add more vague tools

D. Force tool_choice none

55. Format on every file save, deterministically, in Claude Code. Use:

A. A prompt reminder

B. A hook bound to the save lifecycle event

C. A random slash command

D. A bigger context window

56. Chain-of-thought helps most on:

A. Simple lookups

B. Multi-step math/logic/analysis

C. Tasks where reasoning is harmful

D. Reducing latency

57. Urgent chats plus a huge nightly analytics job. Sensible split?

A. Everything real-time

B. Chats real-time; analytics via Batch API

C. Batch the chats

D. One giant prompt for both

58. Incoherent synthesis from five workers. Most direct fix?

A. Give the orchestrator a structured synthesis step with a schema

B. Bigger worker outputs

C. Collapse to one monolith

D. Workers write freely to a shared file

59. A custom repeatable team operation should be exposed as:

A. A Slack message

B. A reusable custom slash command in the project

C. A retyped one-off prompt

D. A comment in unrelated code

60. Keeping a long-running agent accurate and affordable at scale is best achieved by:

A. Never summarizing, caching, or retrieving

B. Caching stable prefixes, retrieving relevant context, compacting history, batching latency-tolerant work

C. Max temperature and tokens every call

D. Sending the full transcript maximally each turn

ANSWER KEY & RATIONALES

Mock Exam A

1: B

Money-critical rules must be enforced in code, not requested in a prompt. Only a deterministic gate guarantees the threshold.

2: B

With a small, stable outcome set, a single classify-then-dispatch workflow is simplest and most reliable.

3: B

Subagent contexts are isolated; only explicitly returned content reaches the orchestrator.

4: B

Independent subtasks run concurrently to minimize latency.

5: B

Project memory is shared and versioned with the code.

6: B

Naming the tool forces exactly that tool; 'any' only forces some tool.

7: B

Structured, observable errors let the loop retry, switch, or stop gracefully.

8: B

Finite windows require compaction to stay focused.

9: B

Batch trades turnaround for a large discount on latency-tolerant work.

10: B

Tool-schema structured output plus code validation is the strongest guarantee of shape.

11: B

Hooks are deterministic lifecycle gates.

12: B

Termination is a property of the controlling code.

13: A

Caching reuses a stable prefix without reprocessing it each request.

14: B

stop_reason reports why generation ended, including max-token truncation.

15: B

Removing the capability at the environment layer is deterministic and unbypassable.

16: A

Examples communicate mapping and format that prose struggles to specify.

17: B

Irreversible actions need deterministic pre-execution checks.

18: B

User-level config is per-developer and not shared.

19: B

Validation gates between chain links catch contract violations early.

20: B

'any' compels a tool call but not which one.

21: B

Bounded backoff retries handle transient faults without hammering or looping forever.

22: B

Separate reasoning and answer sections keep accuracy while allowing clean extraction.

23: B

MCP standardizes connections to external systems.

24: B

Enforce the check before the point of no return, with human approval.

25: B

Structural constraints (schema or prefill) eliminate stray preamble.

26: B

Explicit checkable stop conditions belong in the loop code.

27: B

The model selects tools from their descriptions and schemas.

28: B

max_tokens is the hard bound; stop_reason lets you handle truncation.

29: B

Idempotency makes repeated identical calls safe.

30: A

Routing dispatches by request type for quality and cost.

31: B

Headless mode is built for automation and CI.

32: B

Retrieval supplies only the relevant slice, staying within limits.

33: B

Ownership of control flow is the defining difference.

34: A

Project memory is versioned and shared; secrets would leak.

35: A

Force a tool for required turns and improve descriptions so tools are recognized.

36: B

Truncation means more remains; continue or enlarge the budget.

37: B

Clear delimiters improve reliability, especially in long prompts.

38: B

Parallel calls cut sequential round-trips.

39: B

Decomposition buys testability and maintainability.

40: B

Low temperature reduces sampling variance.

41: B

The evaluator scores and gives feedback; the generator revises.

42: B

Refine the hook to keep the gate without false blocks.

43: B

Persistent behavior like tone belongs in the system prompt.

44: A

Isolated, well-scoped subtasks (e.g., code review) suit subagents.

45: B

Tools act; resources are data the client can read.

46: B

Scoped contexts and deliberate summarization restore focus.

47: B

Numeric caps are deterministic and live in orchestration code.

48: B

The loop must run the tool and feed back the result.

49: B

Understand the layered precedence rather than assuming a fixed winner.

50: B

Retrieval supplies the relevant slice, improving focus and cost.

51: B

Prefilling nudges format, e.g., starting with '{' for JSON.

52: A

Structured errors enable graceful handling instead of crashes.

53: A

Low temperature yields consistent, repeatable outputs.

54: B

Precise descriptions drive correct selection.

55: B

Hooks run commands deterministically on lifecycle events.

56: B

Intermediate steps cut errors on multi-step problems.

57: B

Match mode to latency need: real-time for chats, batch for analytics.

58: A

A schema-driven synthesis step disciplines the merge.

59: B

Custom commands make workflows reusable and consistent.

60: B

Combining the levers keeps quality high and cost low.

Readiness Self-Assessment

Before you register, use this honest self-assessment. Score each domain from the practice sets and the mock exam, then read the guidance below. The real exam passes at 720 on a 100–1000 scale — roughly seventy-two percent — but a candidate who is merely scraping that line on practice tests should keep studying, because exam-day pressure and unfamiliar phrasing typically cost a few points.

90%+ across all five domains	You are ready. Register with confidence and do a light review the day before.
80–89% overall, no domain below 75%	Nearly there. Target your two weakest domains for a focused week, then register.
72–79% overall	Borderline. Do not register yet; the margin is too thin for exam-day variance. Rework missed questions until you can explain every distractor.
Below 72% overall, or any domain below 60%	Not ready. Return to the study guide for the weak domains, then re-attempt the mock exam cold before considering registration.

Conclusion

Practice is not rehearsal for understanding; practice is how understanding is built. If you have worked through these questions with discipline — committing to an answer before reading the explanation, mining every distractor for the misconception it encodes, and returning to your weak domains rather than your strong ones — then you have done more than prepare for an exam. You have internalized the habits of mind that the certification is meant to certify.

Notice how often the same principle surfaced across all five domains, in dozens of different costumes: for anything that touches money, security, deletion, or safety, a competent architect enforces the constraint in code, not in a prompt. A gate, a permission, a cap, a validation layer, a human-in-the-loop before the irreversible step. Prompts shape behavior; code guarantees it. If you remember nothing else from this book, remember that, because it is the spine of professional judgment in this field and the answer to a startling share of the hardest questions.

The second thread was proportion: match the amount of machinery to the amount of genuine uncertainty. A stable classification does not need an autonomous agent; independent subtasks do not need a sequential chain; a demo does not need production credentials. The mark of

maturity is not reaching for the most powerful pattern but choosing the simplest one that is sufficient, and reserving complexity for problems that truly require it.

When you sit the real exam, read each scenario as a working architect would. Ask what could go wrong, where the irreversible actions are, and which constraints must be guaranteed rather than hoped for. Trust the preparation you have done. And whatever the score report says, carry these principles into the systems you build, for the credential is a beginning, not a destination — a signal to others that you have started down a road whose real rewards lie in the reliable, humane, and well-reasoned systems you will go on to create.

And God knows best.

References

Official Anthropic Resources

- Anthropic — Claude Certification Program overview and exam pages (anthropic.com).
- Anthropic Academy — free preparatory courses (anthropic.skilljar.com).
- Anthropic Documentation — Messages API, tool use, and structured output guides (docs.claude.com).
- Anthropic — Claude Code documentation: configuration, hooks, permissions, headless mode.
- Anthropic — Prompt engineering overview and best practices.

Agentic Systems & Orchestration

- Anthropic Engineering — 'Building effective agents': workflows vs. agents, chaining, routing, parallelization, orchestrator–workers, evaluator–optimizer.
- Anthropic — guidance on agent loops, stop conditions, and guardrails.
- Anthropic — multi-agent research system engineering notes and context-isolation considerations.

Tools & the Model Context Protocol

- Model Context Protocol — specification and introduction (modelcontextprotocol.io).
- Anthropic Documentation — tool_choice modes (auto, any, named) and tool result handling.
- Anthropic Documentation — designing tool descriptions and input schemas; structured error handling.

Reliability, Context & Cost

- Anthropic Documentation — prompt caching for stable prefixes.
- Anthropic Documentation — the Message Batches API: turnaround and pricing trade-offs.
- Anthropic Documentation — the stop_reason field and handling truncation.
- Anthropic Documentation — long-context strategies, retrieval, and summarization.

About the Author

Dr. Fadhl Mohammed Alakwaa is a researcher in computational biology and bioinformatics at the University of Michigan, in the Department of Internal Medicine, Division of Nephrology. His research applies machine learning and the analysis of genomic and metabolomic data in the service of precision medicine.

Alongside his research, Dr. Fadhl maintains a bilingual digital library that brings the sciences of artificial intelligence and modern technology closer to Arabic and English readers alike. This practice-exam volume is a companion to his comprehensive guide to the Claude Certification Program, written in the conviction that mastering the new tools of this era is a door of opportunity open to anyone willing to prepare deliberately — regardless of country or university.

Publications

1. The Claude Certification Program: A Comprehensive Guide to Professional Accreditation (2026)
2. The Claude Certification Program: Practice Exams (2026)
3. The End of the Universe in the Holy Qur'an — A Comprehensive Scientific Analysis (2026)

4. Scientific Inimitability in the Qur'an — A Genomics Researcher's View
5. How Large Language Models Work
6. The Impact of Large Language Models on Scientific Research
7. Mastering Agentic Artificial Intelligence
8. AI Agents in Bioinformatics
9. Artificial Intelligence in Medicine
10. Claude Code
11. The Future of Bioinformatics
12. A Guide for the New Yemeni Immigrant to America
13. Professional Licensing Examinations in America
14. How to Succeed Academically at American Universities
15. Silicon Valley

Published by the Fadhl Alakwaa Digital Library, First Edition 2026. Typeset in Amiri, A5 format. This book was produced with the assistance of Claude Opus 4.8 (Anthropic).

About This Book

This companion to the comprehensive guide puts your knowledge to the test. Inside are 138 original, exam-style questions — 78 arranged by the five domains of the Architect — Foundations blueprint, plus a full-length 60-question mock exam — every one with a fully explained answer. You will not merely learn which option is right; you will learn why each wrong option is wrong. A readiness self-assessment helps you decide when you are truly prepared to register.

About the Author

Dr. Fadhl Mohammed Alakwaa is a researcher in computational biology and bioinformatics at the University of Michigan, and the author of a bilingual digital library bringing AI and technology closer to readers in both Arabic and English.

This book was produced with the assistance of Claude Opus 4.8 (Anthropic).
أُنتج هذا الكتاب بمساعدة نموذج الذكاء الاصطناعي Claude Opus 4.8 من شركة Anthropic.